SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO* (BL3, BL6)

Assessment Tool Weightage: 40%

Code of Conduct:

I _C.SANGITA(192373059) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sangita.c

Annexure A

Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.

4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results

Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Question 1: Secure Email Transmission using S/MIME

1. Secure Email Transmission using S/MIME

Aim

To simulate secure email transmission using S/MIME by implementing message encryption, digital signature generation, and signature verification. The program demonstrates how confidentiality, integrity, authentication, and non-repudiation are achieved.

Python Program

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.fernet import Fernet
# Generate RSA key pairs for sender and receiver
sender_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
sender_public_key = sender_private_key.public_key()
receiver_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
receiver_public_key = receiver_private_key.public_key()
# Original email message
message = b"Confidential email: Meeting is scheduled at 10 AM."
# Generate symmetric session key
session_key = Fernet.generate_key()
cipher = Fernet(session_key)
# Encrypt the email using symmetric encryption
encrypted_message = cipher.encrypt(message)
# Encrypt the session key using receiver's public RSA key
encrypted_session_key = receiver_public_key.encrypt(
```

```
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Generate digital signature using sender's private key
signature = sender_private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("Original Message:")
print(message.decode())

print("\nEncrypted Message:")
print(encrypted_message)

print("\nDigital Signature Generated Successfully.")

# Receiver decrypts the session key
decrypted_session_key = receiver_private_key.decrypt(
    encrypted_session_key,
```

```
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Receiver decrypts the email
receiver_cipher = Fernet(decrypted_session_key)
decrypted_message = receiver_cipher.decrypt(encrypted_message)

print("\nDecrypted Message:")
print(decrypted_message.decode())

# Verify digital signature
try:
    sender_public_key.verify(
        signature,
        decrypted_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("\nSignature Verification: SUCCESS")
    print("The message is authentic and has not been modified.")
except Exception:
    print("\nSignature Verification: FAILED")
```

Explanation and Analysis

1. Key Generation:
   RSA public and private key pairs are generated for both sender and receiver. The private keys are kept secret while public keys can be distributed.
2. Message Encryption:
   The email message is encrypted using a symmetric Fernet session key. Symmetric encryption provides efficient confidentiality for the actual email content.
3. Session Key Protection:
   The symmetric session key is encrypted using the receiver's RSA public key. Only the receiver possessing the corresponding private key can recover the session key.
4. Digital Signature:
   The sender creates a digital signature using the sender's private RSA key. This proves that the message originated from the claimed sender.
5. Signature Verification:
   The receiver uses the sender's public key to verify the signature. If verification succeeds, the message has not been altered after signing.
6. Confidentiality:
   Encryption prevents unauthorized users from reading the email. Even if the encrypted email is intercepted, the original message cannot be directly obtained.
7. Authentication and Integrity:
   The digital signature provides sender authentication and detects modifications. A changed message will cause signature verification to fail.
8. Conclusion:
   The simulation demonstrates the major security properties of S/MIME: confidentiality, authentication, integrity, and non-repudiation. Hybrid encryption makes the process both secure and efficient.

Question 2: PGP-Based File Encryption and Decryption

Aim

To develop a Python application that simulates PGP-based file encryption and decryption. The program demonstrates key generation, key distribution, encryption, decryption, and message authentication.

Python Program

from cryptography.hazmat.primitives.asymmetric import rsa, padding

from cryptography.hazmat.primitives import hashes

```
from cryptography.fernet import Fernet

# Generate sender and receiver RSA key pairs
sender_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
sender_public_key = sender_private_key.public_key()

receiver_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
receiver_public_key = receiver_private_key.public_key()

# File data to be protected
file_data = b"PGP Secure File: Confidential Project Report."

print("Original File Data:")
print(file_data.decode())

# Generate symmetric session key
session_key = Fernet.generate_key()
cipher = Fernet(session_key)

# Encrypt file data
encrypted_file = cipher.encrypt(file_data)

# Encrypt session key using receiver's public key
```

```
encrypted_session_key = receiver_public_key.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Generate digital signature
signature = sender_private_key.sign(
    file_data,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nEncrypted File:")
print(encrypted_file)

print("\nSession Key Encrypted Using Receiver Public Key.")
print("Digital Signature Generated.")

# Receiver decrypts the session key
decrypted_session_key = receiver_private_key.decrypt(
    encrypted_session_key,
    padding.OAEP(
```

```
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Decrypt the file
receiver_cipher = Fernet(decrypted_session_key)
decrypted_file = receiver_cipher.decrypt(encrypted_file)

print("\nDecrypted File Data:")
print(decrypted_file.decode())

# Verify signature
try:
    sender_public_key.verify(
        signature,
        decrypted_file,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\nSignature Verification: SUCCESS")
    print("File is authentic and unchanged.")

except Exception:
```

```
    print("\nSignature Verification: FAILED")
```

Explanation and Analysis

1. Key Generation:
   RSA public and private keys are generated for the sender and receiver. The private keys are securely maintained by their respective owners.

2. Session Key Generation:
   A random symmetric key is generated for encrypting the file. Symmetric encryption is much faster than using asymmetric encryption for the complete file.

3. File Encryption:
   The file contents are encrypted using the symmetric session key. This ensures that unauthorized users cannot read the protected file.

4. Key Distribution:
   The session key is encrypted using the receiver's public RSA key. Therefore, only the receiver's private key can recover the session key.

5. Digital Signature:
   The sender signs the original file using the sender's private key. The signature provides authentication and integrity.

6. File Decryption:
   The receiver first uses the receiver's private RSA key to decrypt the session key. The recovered session key is then used to decrypt the file.

7. Authentication:
   The receiver verifies the digital signature using the sender's public key. Successful verification confirms that the file came from the expected sender and was not modified.

8. Efficiency and Security:
   PGP uses a hybrid encryption approach, combining fast symmetric encryption for data with asymmetric encryption for key protection. This provides strong security while remaining efficient for large files.

Conclusion

The program successfully demonstrates the basic principles of PGP-based secure file communication. Symmetric encryption provides confidentiality, RSA protects the session key, and digital signatures provide authentication and integrity.